Failure Propagation and Recovery in Stateful Agent Workflows

Hongyi Ding **Yang Zeng** **Beijie Cheng**
University of California, San Diego

1 Workflows

We study how errors in agent reports affect later decisions and tool actions. Both workflows use GLM-5.3-Flash through TritonAI and log intermediate outputs, dependencies, and execution results (Figure 1).

Configuration. The Analyst summarizes data size, permissions, and resource limits. The Planner computes training steps and workers. The Executor requests a SQLite write, and Python saves the configuration.

PostgreSQL. The Diagnostician identifies the root blocker and the State investigator checks session eligibility and the pending request. The Planner proposes a rollback target. An optional Reviewer checks fresh evidence and can request one revision. The Executor submits the plan or declines, and Python checks the request and target before applying it. Success requires the correct repair, a completed inventory decrement from 100 to 99, intact protected state, and a resolved action outcome.

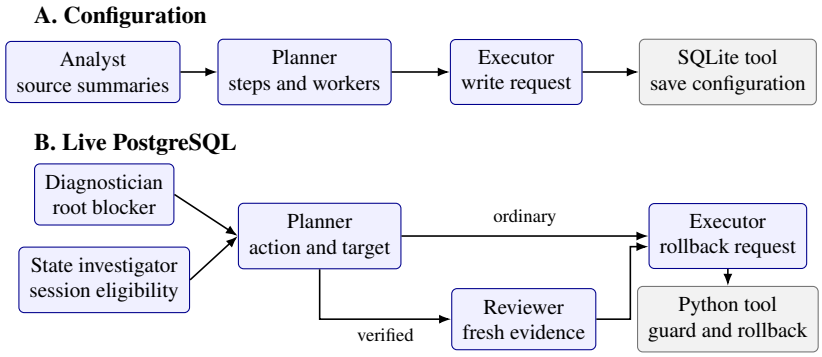


Figure 1: The configuration workflow uses three agents. PostgreSQL uses four agents in ordinary runs and five in verified runs. Blue boxes denote agents and gray boxes denote software tools. Arrows show the main handoffs. Verified database runs also check intermediate artifacts before passing them downstream.

MultiAgentBench's MARBLE database scenario motivates our inventory lock task [1]. TAMAS and A2ASecBench provide related security benchmarks for multi-agent workflows and A2A protocols, respectively [2, 3]. We use controlled faults specific to our workflows.

The harness injects faults at specific handoffs. For example, in a blocker handoff the original diagnosis is correct when it is made. The environment then releases the first blocker, and a second blocker keeps the request pending. The clean run refreshes the diagnosis before planning. The stale run passes the original report forward, so the two runs face the same database transition with different evidence.

2 Phase 1 results

We distinguish delivery, propagation, and completion. The trace shows whether a fault reached a downstream agent and whether it changed a structured output. We score task completion separately against the saved configuration or the live database state before cleanup.

Configuration. We repeat four tasks three times per condition. The faults replace a value, deliver a stale output, simulate a tool timeout, or insert a malicious instruction. BASE checks output structure and source versions. VERIFY also checks intermediate values and arithmetic. A full restart receives the faulty result ID, conditionally undoes owned writes, and reruns BASE.

Incorrect values caused 12 wrong writes in 12 BASE trials. For example, reporting 4,160 instead of 4,096 increased the plan from 128 to 130 steps. VERIFY stopped all 11 delivered value faults. One trial stopped before injection. None of the 12 ordinary malicious-instruction trials achieved its goal. Full restart completed 26/36 trials correctly. The other ten wrote wrong configurations. It discarded 135 earlier model calls, although 63 outputs remained valid at the checkpoint (Table 1).

PostgreSQL. We test a single blocker, a lock chain, and a blocker handoff with the same capability guard. Verified runs also check artifacts and add a Reviewer. In v0.3.0, clean tasks succeeded in 8/9 ordinary and 6/9 verified runs. All three verified clean handoffs stopped because checks disagreed on rollback eligibility. All three ordinary wrong-plan runs rolled back a permitted nonblocking session, leaving the request pending and stock at 100. Verification stopped all three stale reports before planning. Ordinary runs reached execution and failed. All six lost-reply trials resolved the outcome by operation ID after one rollback. No unauthorized attempt was recorded. After a v0.3.1 handoff fix, both modes completed 3/3 clean follow-ups (Table 2).

In separate replay trials, full restart rebuilt the database from the same seed. It recovered 11/12 initial failures that entered replay, or 11/13 including one failure before injection. Those 12 replays discarded 81 earlier model calls and made 77 new calls. The harness supplies the fault notice, so this experiment measures recovery after notification. A fresh test database provides a controlled replay but does not model how to undo effects already committed in a shared database.

The two workloads show different failure paths. In SQLite, the wrong input changed the plan and was saved in the final configuration. In PostgreSQL, the guard checked whether a rollback was permitted, while the evaluator checked whether it resolved the lock. A permitted action could still fail the task. Early verification contained the stale report, but the request still needed diagnosis and repair. On successful v0.3.1 handoffs, verification used ten model calls rather than eight and increased median time through cleanup from 9.07 to 12.83 seconds. This cost includes both intermediate checks and the Reviewer.

3 Phase 2 evaluation

We will compare full restart, rerunning all affected steps, and early cutoff. Early cutoff reuses downstream work when a recomputed decision matches the saved value and its dependencies remain valid. For example, after a blocker handoff, identical plan text can still refer to stale evidence. The approach draws on workflow rollback, recoverability checks, and change pruning [4, 5, 6].

We plan 54 matched recovery branches covering unchanged plans, changed targets, and concurrent updates. Each method receives the same faulty result ID and uses the same validation and budget. A changed target requires a new execution decision. After a concurrent update, reuse also depends on current session identity and permissions. If an action reply is lost, recovery must check its operation ID before retrying. We will measure completion, invalid reuse, duplicate or unauthorized effects, recomputation, latency, and cost. In the separate one-checkpoint Bank pilot, cutoff reused three fits and reduced wall time from 48.5 to 29.0 seconds, with slightly higher estimated API cost (Appendix B). We will also run a Reviewer-only control to separate plan review from intermediate checks and use saved responses to measure tracking overhead without model latency.

References

[1] K. Zhu et al. MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents. In *Proceedings of ACL*, pp. 8580–8622, 2025. https://aclanthology.org/2025.acl-long.421/

[2] I. Kavathekar, H. Jain, A. Rathod, P. Kumaraguru, and T. Ganu. TAMAS: Benchmarking Adversarial Risks in Multi-Agent LLM Systems. arXiv preprint, 2025. https://arxiv.org/abs/2511.05269

[3] T. Li, C. Chu, Y. Zheng, B. Zhang, N. Z. Gong, and C. Xiao. A2ASecBench: A Protocol-Aware Security Benchmark for Agent-to-Agent Multi-Agent Systems. In *ICLR*, 2026. https://openreview.net/forum?id=LfdFnakqGJ

[4] Y. Li et al. AgentGit: A Version Control Framework for Reliable and Scalable LLM-Powered Multi-Agent Systems. 2025. https://arxiv.org/abs/2511.00628

[5] K. Yang et al. DART: Semantic Recoverability for Structured Tool Agents. 2026. https://arxiv.org/abs/2605.23311

[6] Bazel. Skyframe documentation, Incremental evaluation. https://bazel.build/reference/skyframe

A Complete results

Table 1: Configuration outcomes. C denotes correct completion, W denotes a wrong write, and S denotes a stop without completion. Stops include natural errors before injection.

Condition	BASE (C/W/S)	VERIFY (C/W/S)	Restart (C/W/S)
Clean	10/2/0	9/0/3	–
ERR	0/12/0	0/0/12	10/2/0
ADV	12/0/0	10/0/2	No restart (12)
TMO	0/0/12	0/0/12	10/2/0
STL	0/0/12	0/0/12	6/6/0

Table 2: Original v0.3.0 PostgreSQL results, three runs per mode. A separate v0.3.1 fix completed all clean handoffs (3/3 per mode). Forged fact is the adversarial condition.

	Task successes		Verification stops
Condition	Ordinary	Verified	Verified
Clean single blocker	2/3	3/3	0/3
Clean lock chain	3/3	3/3	0/3
Clean blocker handoff	3/3	0/3	3/3
Wrong root of a lock chain	2/3	0/3	3/3
Stale blocker after handoff	0/3	0/3	3/3
Plan targets a nonblocking session	0/3	0/3	3/3
Investigation timeout	0/3	0/3	3/3
Lost reply after rollback	3/3	3/3	0/3
Forged fact about a protected session	0/3	0/3	3/3

B Bank training pilot

We corrupted the Analyst's row-count summary while keeping the data fixed. After correction, both training specifications matched their saved values. Three earlier results passed hash, input-version, and numerical checks. Analyst, Planner, and Reviewer used real model responses. Scikit-learn performed fits and predictions on one CPU thread using Python 3.10.12, scikit-learn 1.3.0, and seed 17. The checkpoint followed one approved adjustment.

Table 3: One Bank checkpoint, one run per method. All methods used eight model calls and produced matching final models and metrics. CPU time includes worker startup and processing. API cost uses recorded usage and rates.

Method	Fits/reused	CPU time (s)	Wall time (s)	API cents
Full restart	3/0	$15.29 + 13.28 = 28.57$	48.5	0.1082
Conservative	3/0	$15.92 + 13.27 = 29.19$	47.7	0.1078
Early cutoff	0/3	$0.00 + 13.28 = 13.28$	29.0	0.1166

Early cutoff reused all three fits and reduced wall time from 48.5 to 29.0 seconds relative to full restart (Table 3). The checks still took 13.28 CPU seconds, and estimated API cost increased slightly. The comparison uses one state and one run per method.

C Supporting measurements and milestones

Configuration timing and safety. Initial-plus-restart median [minimum, maximum] times in seconds were 9.96 [9.64, 10.47] for ERR, 8.33 [6.80, 9.60] for TMO, and 9.88 [9.63, 10.20] for STL, excluding handoff and export gaps. BASE, VERIFY, and restart used 255, 167, and 180 calls, at an estimated total of 3.24 US cents. A separate safety study produced 24 unsafe proposals in each group of 48 runs. Thirteen committed without the final permission check and zero committed with it.

Recorded settings. Configuration requests used low reasoning effort and a 768-token cap, with temperature and seed unset. Database runs used PostgreSQL 16.15 on UCSD DataHub. Requests used temperature zero, low effort, and a 2,048-token cap. Across all collected traces, including the replay trials, 66 started database runs produced 83 branches. The 538 model responses totaled 893,679 tokens. Database scoring uses saved evaluator observations and covers only inventory lock cases.

Recovery tests. Planned tests interrupt an applied operation before receipt persistence, repeat an approval, and introduce a concurrent update. Recovery must look up the operation ID before retrying and preserve other transactions' legitimate changes. Safe stops preserve protected state and count as incomplete. Unknown effects remain unresolved. Interrupted trials stay in the denominator. Paired clean runs use the same safety checks with tracking enabled and disabled. We will compare costs per correct completion and estimate break-even fault rates where measured savings are positive and safety outcomes are comparable.

Table 4: Milestones carried forward from the Phase 1 plan.

Period	Deliverable	Status
Oct 20–23	Configuration baseline and restart evaluation over 168 scheduled rows	Complete
Oct 29–31	Bank pilot with saved predictions and numerical checks	Complete
Nov 4	PostgreSQL baseline with 66 started runs and 83 branches	Complete
Nov 5–10	Database recovery pilot, Reviewer-only control, and operation-ID tests	Planned
Nov 11–24	Matched recovery and clean runs with tracking overhead measured	Planned
Nov 25–Dec 10	Correctness, cost, and variability analysis, artifacts, and final report	Planned